

软件工程基础第 3 章

软件项目可行性研究

详细学习笔记

2026 年 5 月 28 日

目录

1	可行性研究概述	2
1.1	什么是可行性研究 (<i>Feasibility Study</i>)	2
1.2	可行性研究可能不可行 (<i>Feasibility May Be Negative</i>)	2
1.3	可行性研究的内容 (<i>Content of Feasibility Study</i>)	2
1.4	可行性研究的基本步骤	2
2	系统组织结构分析 (<i>Organizational Structure Analysis</i>)	3
2.1	组织结构图 (<i>Organizational Chart</i>)	3
2.2	业务联系图 (<i>Business Linkage Diagram</i>)	3
2.3	业务功能树的稳定性	3
3	系统处理流程分析 (<i>System Process Flow Analysis</i>)	4
3.1	系统流程图 (<i>System Flowchart</i>)	4
3.2	跨职能流程图 (<i>Cross-Functional Flowchart / Swimlane Diagram</i>)	4
3.3	系统流程图建模基本步骤	4
3.4	系统流程图的作用	4
4	系统数据流分析 (<i>System Data Flow Analysis</i>)	5
4.1	数据流图 (<i>Data Flow Diagram, DFD</i>)	5
4.2	新系统的物理方案	5
5	故障树分析 (<i>Fault Tree Analysis, FTA</i>)	6
5.1	FTA 基本概念	6
5.2	FTA 的自顶向下分析过程	6
5.3	顶事件的确定	6
5.4	系统边界条件	6
5.5	构建故障树	7

5.6	故障树示例	7
5.7	故障树规范化	8
5.8	故障树简化	8
5.9	故障树定性分析——最小割集	8
5.10	求最小割集的方法	9
5.10.1	上行法详细步骤	9
5.10.2	上行法求解示例	9
5.10.3	最小割集分析原则	9
5.11	故障树定量分析 (<i>Quantitative Analysis</i>)	10
5.11.1	定量分析的基本假设	10
5.11.2	单调结构函数 (<i>Monotonic Structure Function</i>)	10
5.11.3	顶事件概率计算	10
5.11.4	近似计算法	11
5.11.5	近似计算示例	11
5.11.6	直接法 (不变化法)	12
5.12	基本事件结构重要度分析 (<i>Structural Importance Analysis</i>)	12
5.12.1	结构重要度系数	12
5.12.2	依据最小割集判断结构重要度 (近似法)	13
5.13	基本事件概率重要度 (<i>Probability Importance</i>)	13
5.13.1	概率重要度计算示例	13
5.14	基本事件关键重要度 (<i>Critical Importance</i>)	14
6	成本估计 (<i>Cost Estimation</i>)	15
6.1	成本构成	15
6.2	GB/T36964-2018 软件开发成本度量规范	15
6.3	成本估算方法分类	15
6.4	自顶向下估算 (<i>Top-Down Estimation</i>)	15
6.5	德尔菲法 (<i>Delphi Method</i>)	15
6.6	由底向上估算 (<i>Bottom-Up Estimation</i>)	16
6.6.1	由底向上估算示例	16
6.7	经验模型估算 (<i>Empirical Model Estimation</i>)	16
6.7.1	静态单变量模型 (<i>Static Single-Variable Model</i>)	17
6.7.2	动态多变量模型——Putnam 模型 (<i>Putnam Model</i>)	17
6.8	COCOMO 模型 (<i>Constructive Cost Model</i>)	17
6.8.1	基本层 COCOMO 模型	18
6.8.2	中间层 COCOMO 模型	18
6.8.3	详细层 COCOMO 模型	18
6.9	COCOMO II 模型	19
6.10	软件开发成本度量规范——估算过程	19
6.11	软件规模估算 (<i>Software Size Estimation</i>)	19

6.12 功能点度量方法	19
6.13 工作量估算与成本计算	20
7 效益分析 (<i>Benefit Analysis</i>)	21
7.1 软件项目效益的两大组成部分	21
7.1.1 经济效益 (<i>Economic Benefits</i>)	21
7.1.2 社会效益 (<i>Social Benefits</i>)	21
7.2 货币的时间价值 (<i>Time Value of Money</i>)	21
7.3 纯收入 (<i>Net Income</i>)	22
7.4 投资回收期 (<i>Payback Period</i>)	22
7.5 投资回收率 (<i>Return on Investment, ROI</i>)	22
7.6 经济效益分析综合示例	22
7.7 效益分析总结	23
7.8 CIMS 项目效益分析实例	23
8 本章要点总结	24

1 可行性研究概述

1.1 什么是可行性研究 (*Feasibility Study*)

软件工程项目可行性研究实质是一次大大压缩和简化了的分析和设计过程，主要在较高层次上以较抽象的方式进行。其目的是在尽可能短的时间内以最小的代价确定该项目是否能够开发，是否值得开发。 [p.4]

可行性研究不是去研发一个软件项目，而是研究该项目能否在给定的资源和给定的时间开发，是否能够开发，是否值得开发。在尽可能早的时间来评估项目的可行性，不仅可以避免资源的浪费，同时也对加快项目的开发有重要影响。 [p.6]

1.2 可行性研究可能不可行 (*Feasibility May Be Negative*)

可行性研究的结论未必总是正的。常见导致不可行结论的原因包括： [p.5]

- 没有认真研究市场和竞争对手情况——竞品分析（客观分析、主观分析）不足；
- 数据来源不准确，没有调查记录——实地考察、文献调研、网络搜索不到位；
- 数据估算不合理、失真——成本/效益分析方法不符合实际；
- 技术方案不合实际——现有技术不能解决项目目标。

1.3 可行性研究的内容 (*Content of Feasibility Study*)

可行性研究主要包含三个方面的分析： [p.7]

1. 技术可行性 (*Technical Feasibility*): 包括待开发系统技术分析、资源有效性分析、风险分析、系统故障分析；
2. 经济可行性 (*Economic Feasibility*): 包括成本估计、效益分析；
3. 社会可行性 (*Social Feasibility*): 就政治意识形态、法律法规、社会道德、民族意识以及系统运行的组织机构或人员等，分析系统能否运行及运行好坏程度。

1.4 可行性研究的基本步骤

要求开发人员根据目前的技术水平和过去的经验来进行可行性研究： [p.9]

1. 系统目标和范围的定义——明确要解决的根本问题、达到目标所需的资源和经费；
2. 对现行系统进行分析研究——建立现有系统的物理模型和逻辑模型（入口：现有系统的组织结构）；
3. 导出新系统的逻辑模型——解决了有关问题；
4. 设计新系统的物理方案——最先进的方案、实用方案、基本方案；
5. 推荐可行的方案——包括推荐理由；
6. 编写可行性研究报告——结论有三种：继续、延期和拒绝。

2 系统组织结构分析 (*Organizational Structure Analysis*)

2.1 组织结构图 (*Organizational Chart*)

系统的组织结构反映了组织内部的组成以及它们之间的关系。内容包括三种视图： [p.14]

- 组织结构图 (*Organizational Chart*): 表示组织结构中的层次关系
- 业务联系图 (*Business Linkage Diagram*): 表示组织与业务的关联
- 业务功能树 (*Business Function Tree*): 表示业务功能的层次分解

在组织结构图中, 每个方框表示一个业务域, 通过层次分解精化, 直至小的工作组或某个个体。例如: 总公司下分为人事处、供销处、生产处、财务处、行政处, 各处分设不同的科室。 [p.15]

2.2 业务联系图 (*Business Linkage Diagram*)

业务联系图反映组织各部分与各项业务之间的联系。其表示法为:

- ×: 表示组织是参加协调该项业务的辅助组织;
- √: 表示该组织是该项业务的相关组织;
- *: 表示该业务是该组织的主要业务;
- 无标记: 表示该组织与该项业务无关。

[p.16]

2.3 业务功能树的稳定性

一般情况下, 组织机构容易发生变化, 且无法预料, 但组织的业务功能是可以发现的。因此业务功能树比组织结构图更稳定, 更适合作为系统分析的基础。 [p.17]

3 系统处理流程分析 (*System Process Flow Analysis*)

3.1 系统流程图 (*System Flowchart*)

系统流程图是描绘物理系统的传统工具，用于描述项目的大概业务处理流程。其基本思想是用图形符号以黑盒子形式描绘系统各部件（如程序、数据库、文档、人工过程等）。 [p.19]

系统流程图表达的是信息在系统中各部件之间流动的情况，而不是对信息进行加工处理的控制过程。 [p.20]

3.2 跨职能流程图 (*Cross-Functional Flowchart / Swimlane Diagram*)

跨职能流程图（又称泳道图）显示流程处理中各步骤之间的关系以及执行它们的职能单位。 [p.21]

两种排版方式：

- **水平跨职能流程图：** 职能单位位于流程图左侧，泳道水平于画布中摆放。水平排版能够凸显处理流程。
- **垂直跨职能流程图：** 职能部门位于流程图上方，泳道垂直于画布摆放。垂直排版更加强调职能单位。

3.3 系统流程图建模基本步骤

根据国标 GB1526-89，系统流程图使用一组标准符号进行描述。 [p.22]

建模基本步骤： [p.23]

1. 首先，建立系统组织结构，并描述组织结构完成哪些业务功能，采用业务联系图或业务功能树描述；
2. 其次，对某个业务处理，考察组织的不同业务功能之间的信息流动，并采用系统流程图描述；
3. 最后，对建立的系统流程图进行完善。

3.4 系统流程图的作用

- 有利于全面了解系统整体业务处理的大概过程；
- 有利于系统分析员、管理员和业务用户相互交流；
- 可用来分析系统业务流程的合理性；
- 是用来构建系统逻辑处理模型的基础。

[p.24]

4 系统数据流分析 (*System Data Flow Analysis*)

4.1 数据流图 (*Data Flow Diagram, DFD*)

将业务功能进一步精化为业务处理，显示每个处理的输入和输出数据对象，指明处理如何变换信息来完成业务功能。 [p.26]

数据流图描述的是系统的逻辑模型，图中没有具体的物理元素，只是描绘数据在系统中的流动和处理情况。这是 DFD 与系统流程图的关键区别。

在可行性研究阶段，采用数据流图来描述数据在系统中的流动和处理情况时，不必详细描述，只须概括地描述高层的数据处理和流动。 [p.27]

4.2 新系统的物理方案

一旦建立了一个系统的数据流图，则开发人员可以 and 用户一道检查现存的处理过程，根据系统的目标和提出的有关问题修订数据流图得到新系统的数据流图，即新系统的逻辑模型。以此为基础，进一步可给出新系统的物理方案，并对每一方案进行可行性研究，最后向用户推荐一种方案。 [p.28]

5 故障树分析 (*Fault Tree Analysis, FTA*)

5.1 FTA 基本概念

在可行性研究阶段,分析由于软件、硬件、环境和人等各种因素造成的系统故障,明确薄弱环节,以便在早期采取相应的措施避免引入故障,确保可信软件开发的可行性。 [p.33]

故障树分析方法(Fault Tree Analysis, 简称 FTA)不需要太多有关系统的具体信息,适合可行性研究阶段。在后续软件开发各阶段,还可以依据更多的信息对故障树不断改进完善。

历史背景:故障树分析方法由美国贝尔电报公司的电话实验室的沃森(Waston)于 1961 年首先提出。 [p.34]

5.2 FTA 的自顶向下分析过程

采用故障树分析法进行系统可信性分析是一个自顶向下的过程: [p.34]

1. 首先,从要分析的特定故障(或事故)开始,称其为**顶事件 (Top Event)**;
2. 接着,建立**系统边界条件 (Boundary Conditions)**,明确原因事件和各因素的调查范围;
3. 然后,逐层分析其发生的原因,直到找出故障的基本原因为止,建立故障树。这些基本原因也称为故障树的**底事件 (Bottom Event)**或**基本事件 (Basic Event)**。基本事件的数据是已知的或者已经有过统计(或实验)的结果;
4. 最后,依据故障树进行**定性分析**和**定量分析**,确定各事件的重要度和事件发生的概率,从而为制定科学、合理的可信性措施提供基本依据。

5.3 顶事件的确定

找出后果严重且较易发生的故障作为顶事件: [p.35]

- 熟悉现有系统,调查收集现有系统故障(或事故)案例,进行故障统计;
- 依据新系统需要和用户要求,设想给定新系统可能会发生的故障;
- 对所调查的故障进行全面分析,从中找出后果严重且较易发生的故障作为顶事件,如对系统安全构成威胁的故障、妨碍完成任务的故障、严重影响经济效益的故障等;
- 如果系统有多个后果严重且较易发生的故障,则需要将这些故障分别作为顶事件。顶事件也可以是指定需要进行分析的故障。

5.4 系统边界条件

所谓系统边界条件是指规定所建故障树何处为止: [p.36]

- 边界条件包括不允许出现的事件、不可能发生的事件(小概率事件)、必然事件(在一定条件下必然发生的事件和必然不发生的事件)、与顶事件发生有关的系统单元初始状态和其他假设等;
- 通过明确系统的内容及其边界范围,在建立故障树时,可以有效确定原因事件和各种因素的调查范围;
- 根据现有系统的经验教训和事故案例,通过统计分析,求解故障发生的概率(频率),以此作为要控制故障的目标值。

表 1: 系统边界条件示例

	不能登录	不能出票
顶事件	不能登录	不能出票
初始状态	系统可以响应访问请求	系统有票，用户已支付
不允许出现的事件	外部因素（如网络不稳定）使系统不能联网	外部因素（如网络不稳定）使系统不能联网
必然事件	无	系统卖不出票

5.5 构建故障树

调查与故障有关的所有原因事件和各种因素： [p.37]

1. 从顶事件起，找出直接原因的事件，将所有直接原因的事件作为输入事件；
2. 依据事件间的逻辑关系，用适当的逻辑门将这些事件关联；
3. 如果输入事件还能进一步分解，则将其看作是输出事件（称其为**中间事件 (Intermediate Event)**），进一步找出直接原因的事件作为输入事件，并依据事件间的逻辑关系用适当的逻辑门将事件关联；
4. 重复上述步骤，逐级分解，直至所有的输入事件不能再分解或不必再分解（称其为**底事件或基本事件**）为止，由此便建立了一颗故障树。

构建注意事项： [p.38]

- 明确建立故障树的目标和范围，有助于缩小故障树的规模；
- 顶事件要有明确的定义，避免多义性，以免使故障树逻辑混乱；
- 从顶事件出发，将顶事件的最直接、充分且必要的原因（而不是基本原因）作为顶事件的输入事件；
- 对不同分支中出现的同一共因事件（引起多个系统故障的事件）必须使用同一事件符号（名称）。

5.6 故障树示例

以下为“系统不能登录”的故障树示例结构： [p.39]

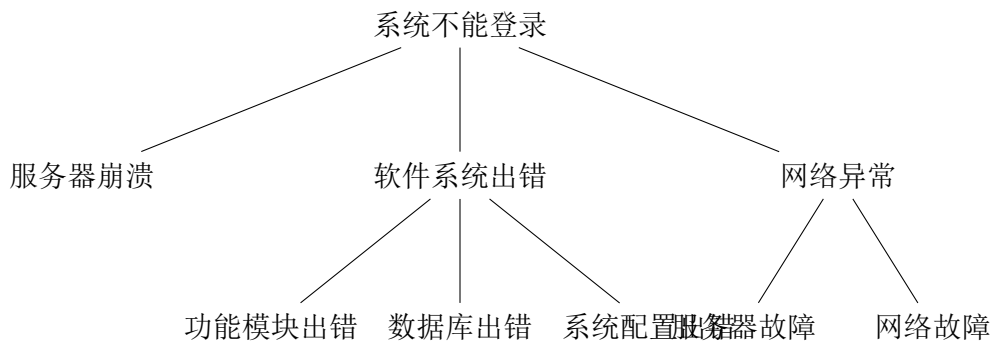


图 1: 系统不能登录故障树（简化示意）

5.7 故障树规范化

规范化故障树是指仅有顶事件、中间事件和基本事件等三类事件，以及与门（AND Gate）、或门（OR Gate）、非门（NOT Gate）三类逻辑门的故障树。规范化操作： [p.41]

- 将未探明事件或当作基本事件或删除；
- 将顺序与门变换为与门；
- 将表决门变换为或门和与门的组合；
- 将禁门变换为与门。

5.8 故障树简化

故障树简化是利用布尔代数去掉逻辑多余事件和逻辑多余门： [p.42]

- 故障树简化可以有效减少故障树的规模，从而减少故障树分析的工作量；
- 故障树简化不是故障分析必要的步骤；
- 故障树不简化不影响故障树定量分析和定性分析的结果。

5.9 故障树定性分析——最小割集

故障树定性分析的目的是求解导致顶事件发生的各种原因以及原因的组合，也就是识别导致顶事件发生的故障模式集合，从而有助于找到消除用户最不希望发生的系统故障的可行方案。 [p.44]

关键概念：

- **割集 (Cut Set):** 故障树中一些基本事件的集合，当这些基本事件同时发生时，顶事件必然发生。
- **最小割集 (Minimal Cut Set):** 若将割集中所包含基本事件中的任意一个去掉就不再是割集了，这样的割集就是最小割集。
- **单点故障 (Single Point of Failure):** 系统中单一原因引起整个系统无法运作的故障。可靠性关键系统不允许有单点故障。
- 找出一阶（即单事件，事件数目即阶数）最小割集。

5.10 求最小割集的方法

求最小割集的方法主要有三种： [p.45]

1. 上行法 (*Bottom-Up Method*): 从故障树的基本事件开始, 依据故障树的实际结构, 逐级向上进行事件的集合运算。
2. 下行法 (*Down-Top Method*): 从顶事件开始向下逐步分解。
3. 布尔代数化简法 (*Boolean Algebra Simplification*)。

5.10.1 上行法详细步骤

上行法从故障树的基本事件开始, 逐级向上进行事件的集合运算： [p.45]

- 遇到与门, 将与门输出事件表示为输入事件的交 (布尔积);
- 遇到或门, 将或门输出事件表示为输入事件的并 (布尔和);
- 遇到非门, 将非门输出事件表示为输入事件的反。

向上层层代入, 并按布尔代数吸收律和幂等律来化简, 将顶事件表示为基本事件的若干积项之和的最简式。这样, 每一积项对应于故障树的一个最小割集。

5.10.2 上行法求解示例

以某故障树为例, 最底层的基本事件为: X_3, X_4, X_5, X_6 。 [p.46-48]

第一步: 向上遇到或门和输出事件 G_2 和 G_3 :

$$G_2 = X_3 \cup X_4 \cup X_5$$

$$G_3 = X_4 \cup X_5 \cup X_6$$

第二步: 进一步往上, 遇到与门和输出事件 G_1 :

$$\begin{aligned} G_1 &= G_2 \cap G_3 \\ &= (X_3 \cup X_4 \cup X_5) \cap (X_4 \cup X_5 \cup X_6) \\ &= (X_3 \cap X_6) \cup X_4 \cup X_5 \end{aligned}$$

第三步: 进一步往上, 遇到或门和输出事件顶事件 T :

$$\begin{aligned} T &= X_1 \cup G_1 \cup X_2 \\ &= X_1 \cup (X_3 \cap X_6) \cup X_4 \cup X_5 \cup X_2 \end{aligned}$$

结论: 故障树的全部最小割集为: $\{X_1\}, \{X_2\}, \{X_3, X_6\}, \{X_4\}, \{X_5\}$ [p.48]

5.10.3 最小割集分析原则

阶数: 最小割集含基本事件的数目。 [p.48]

分析原则 (按重要性排序):

1. 阶数越小的最小割集越重要;
2. 在低阶最小割集中出现的基本事件比高阶最小割集中的基本事件重要;
3. 在最小割集阶数相同的条件下, 在不同最小割集中重复出现的次数越多的基本事件越重要;
4. 最小割集数目越多, 系统的风险性越大。

5.11 故障树定量分析 (*Quantitative Analysis*)

在通过故障树定性分析计算得到全部最小割集后，如果能够确定故障树中各个基本事件发生的概率，则可以对故障树进行定量分析。 [p.50]

定量分析主要内容：

- 确定基本事件发生的概率；
- 利用基本事件的发生概率计算每个最小割集的发生概率；
- 利用最小割集发生的概率计算出顶事件的发生概率；
- 分析基本事件的结构重要度、概率重要度、相对概率重要度（或称为关键重要度）。

5.11.1 定量分析的基本假设

- 基本事件之间相互独立；
- 基本事件（顶事件）都只考虑两种状态：发生或不发生（故障或正常）；
- 假定故障分布为指数分布；
- 系统是单调关联系统——系统所含单元都与系统有关且系统对应的结构函数是单调非减的。

[p.51]

5.11.2 单调结构函数 (*Monotonic Structure Function*)

单调结构函数反映了系统单元状态的改善不会使系统变坏。对给定的单元状态，系统的状态 $\Phi(X)$ 表示如下： [p.52]

$$\Phi(X) = \begin{cases} 1, & \text{若系统正常} \\ 0, & \text{若系统故障} \end{cases}$$

则称 $\Phi(X)$ 为系统的结构函数 (*Structure Function*)。若对任意的 $X \leq Y$ 有 $\Phi(X) \leq \Phi(Y)$ ，则称 $\Phi(X)$ 是单调结构函数。

5.11.3 顶事件概率计算

根据系统结构函数和基本事件的发生概率，应用逻辑与（AND）和逻辑或（OR）的概率计算公式来计算。 [p.53-54]

由于基本事件相互独立，最小割集 C_i 发生的概率等于最小割集中各基本事件发生概率之积：

$$P(C_i) = \prod_{X_j \in C_i} P(X_j)$$

互斥情况：如果各最小割集发生是相互独立（互斥）的事件，则顶事件 T 发生的概率等于各最小割集发生的概率之和：

$$P(T) = \sum_{i=1}^m P(C_i)$$

非互斥情况（全概率法）：如果各最小割集发生不是相互独立的事件，则：

$$\begin{aligned}
 P(T) = & \sum_{i=1}^m P(C_i) - \sum_{i=1}^{m-1} \sum_{j=i+1}^m P(C_i C_j) \\
 & + \sum_{i=1}^{m-2} \sum_{j=i+1}^{m-1} \sum_{k=j+1}^m P(C_i C_j C_k) \\
 & + \cdots + (-1)^{m-1} P(C_1 C_2 \cdots C_m)
 \end{aligned}$$

5.11.4 近似算法

如果故障树的最小割集数 m 特别大，计算顶事件概率会出现**组合爆炸 (Combinatorial Explosion)** 问题。实际工程中统计得到的基本事件概率往往不准确，因此常常采用近似法：[p.55]

$$\begin{aligned}
 P_1 &= \sum_{i=1}^m P(C_i) \\
 P_2 &= \sum_{i=1}^{m-1} \sum_{j=i+1}^m P(C_i C_j)
 \end{aligned}$$

P_1 是顶事件发生概率 $P(T)$ 的上界， $P_1 - P_2$ 是 $P(T)$ 的下界。可以将 P_1 或 $P_1 - P_2$ 作为 $P(T)$ 的近似值，分别称为**一阶近似**和**二阶近似**。

5.11.5 近似计算示例

假定各基本事件发生的概率为： $P(X_1) = P(X_2) = 0.1, P(X_3) = 0.2, P(X_4) = P(X_5) = 0.3, P(X_6) = 0.1$ 。故障树的全部割集为： $\{X_1\}, \{X_2\}, \{X_3, X_6\}, \{X_4\}, \{X_5\}$ [p.56]

$$\begin{aligned}
 P_1 &= P(X_1) + P(X_2) + P(X_3)P(X_6) + P(X_4) + P(X_5) \\
 &= 0.1 + 0.1 + 0.2 \times 0.1 + 0.3 + 0.3 = 0.82
 \end{aligned}$$

$$\begin{aligned}
 P_2 &= P(X_1)P(X_2) + P(X_1)P(X_3)P(X_6) + P(X_1)P(X_4) + P(X_1)P(X_5) \\
 &\quad + P(X_2)P(X_3)P(X_6) + P(X_2)P(X_4) + P(X_2)P(X_5) \\
 &\quad + P(X_3)P(X_6)P(X_4) + P(X_3)P(X_6)P(X_5) + P(X_4)P(X_5) \\
 &= 0.1 \times 0.1 + 0.1 \times 0.2 \times 0.1 + 0.1 \times 0.3 + 0.1 \times 0.3 \\
 &\quad + 0.1 \times 0.2 \times 0.1 + 0.1 \times 0.3 + 0.1 \times 0.3 \\
 &\quad + 0.2 \times 0.1 \times 0.3 + 0.2 \times 0.1 \times 0.3 + 0.3 \times 0.3 \\
 &= 0.236
 \end{aligned}$$

顶事件 T 发生概率的**一阶近似值** $P_1 = 0.82$ ，**二阶近似值** $P_1 - P_2 = 0.82 - 0.236 = 0.584$ 。

5.11.6 直接法（不变化法）

也可以直接采用集合运算的规则，将两个相交的集合 A 和 B 的并转化成两个不相交的集合的并： [p.57-59]

$$A \cup B = A \cup (\bar{A} \cap B)$$

$$P(A \cup B) = P(A) + ((1 - P(A)) \cdot P(B))$$

使用直接法计算中间事件概率（以故障树为例）：

$$\begin{aligned} P(G_3) &= P(X_4) + (1 - P(X_4))(P(X_5) + (1 - P(X_5))P(X_6)) \\ &= 0.3 + (1 - 0.3)(0.3 + (1 - 0.3) \times 0.1) = 0.559 \end{aligned}$$

$$P(G_2) = P(X_3) + (1 - P(X_3))(P(X_4) + (1 - P(X_4))P(X_5)) = 0.608$$

$$\begin{aligned} P(G_1) &= P(X_3)P(X_6) + (1 - P(X_3)P(X_6))(P(X_4) + (1 - P(X_4))P(X_5)) \\ &= 0.2 \times 0.1 + (1 - 0.02)(0.3 + 0.7 \times 0.3) = 0.5198 \end{aligned}$$

顶事件 T 的发生概率：

$$\begin{aligned} P(T) &= P(X_1) + (1 - P(X_1))(P(X_2) + (1 - P(X_2)) \\ &\quad \times (P(X_3)P(X_6) + (1 - P(X_3)P(X_6))(P(X_4) + (1 - P(X_4))P(X_5)))) \\ &= 0.1 + (1 - 0.1)(0.1 + (1 - 0.1) \\ &\quad \times (0.2 \times 0.1 + (1 - 0.2 \times 0.1)(0.3 + (1 - 0.3) \times 0.3))) \\ &= 0.611038 \end{aligned}$$

直接法计算出的顶事件的概率比一阶近似的（0.82）结果小，而比二阶近似（0.584）的结果大。 [p.59]

5.12 基本事件结构重要度分析 (*Structural Importance Analysis*)

基本事件结构重要度分析不考虑基本事件发生的概率，仅从故障树的结构上分析各基本事件的发生对顶事件发生的影响程度。 [p.61]

主要有两种方法：

1. **精确方法：**计算出各基本事件结构重要度系数，然后按系数由大到小排列各基本事件结构重要顺序。但随着基本事件的增加，计算量呈指数增长。
2. **近似方法：**依据最小割集判断各基本事件结构重要顺序。既有足够的精度，又不过于复杂。一般情况下，当故障树规模较大时（基本事件的个数超过 18 个），建议采用近似方法。

5.12.1 结构重要度系数

第 i 个基本事件 X_i 的结构重要度系数： [p.62]

$$I_i^\phi = \frac{1}{2^{n-1}} \sum [\phi(X_1, \dots, X_{i-1}, 1, X_{i+1}, \dots, X_n) - \phi(X_1, \dots, X_{i-1}, 0, X_{i+1}, \dots, X_n)]$$

其中 ϕ 为故障树的结构函数。第 i 个基本事件 X_i 的结构重要度系数反映了基本事件 X_i 对系统故障的出现有重要作用。

5.12.2 依据最小割集判断结构重要度（近似法）

依据最小割集，采用四条原则来判断基本事件结构重要性： [p.65]

1. **第一条原则：**一阶（即单事件）最小割集中的基本事件的结构重要度最大；
2. **第二条原则：**仅出现在同一个最小割集中的所有基本事件的结构重要度一样大；
3. **第三条原则：**若两个基本事件仅出现在基本事件个数相等的不同最小割集中，则其结构重要度依出现次数而定，出现次数多的结构重要度大；
4. **第四条原则：**若两个基本事件出现在基本事件个数不相等的不同最小割集中，则：
 - 在各最小割集中出现的次数相等时，在事件数较少的割集中出现的基本事件的结构重要度大；
 - 在事件数较少的割集中出现的次数少和在事件数较多的割集中出现的次数多的基本事件，或其他更为复杂的情况，其结构重要度系数的近似值按公式：

$$I'_i = \sum_{X_i \in C_j} \frac{1}{2^{n_i-1}}$$

5.13 基本事件概率重要度 (*Probability Importance*)

基本事件概率重要度反映了基本事件发生概率变化引起顶事件发生概率变化的程度。可用定量衡量降低各基本事件的发生概率对降低顶事件的贡献大小。 [p.67]

基本事件 X_i 的概率重要度系数：

$$I_p(X_i) = \frac{\partial F(P(X_1), P(X_2), \dots, P(X_n))}{\partial P(X_i)}$$

其中 $F(P(X_1), P(X_2), \dots, P(X_n))$ 为顶事件的概率函数， $P(X_i)$ 为事件 X_i 的概率。

5.13.1 概率重要度计算示例

$$\begin{aligned} F(P(X_1), P(X_2), P(X_3)) &= P(C_1) + P(C_2) - P(C_1 C_2) \\ &= P(X_1)P(X_2) + P(X_2)P(X_3) - P(X_1)P(X_2)P(X_3) \end{aligned}$$

$$I_p(X_1) = \frac{\partial F}{\partial P(X_1)} = P(X_2) - P(X_2)P(X_3) = 0.14$$

$$I_p(X_2) = \frac{\partial F}{\partial P(X_2)} = P(X_1) + P(X_3) - P(X_1)P(X_3) = 0.58$$

$$I_p(X_3) = \frac{\partial F}{\partial P(X_3)} = P(X_2) - P(X_1)P(X_2) = 0.12$$

[p.68]

5.14 基本事件关键重要度 (*Critical Importance*)

基本事件 X_i 关键重要度表示基本事件 X_i 概率变化率对顶事件概率变化率的影响程度。
[p.69]

基本事件 X_i 的关键重要度:

$$I_c^p(X_i) = \frac{P(X_i)}{P(T)} \cdot I_p(X_i)$$

6 成本估计 (*Cost Estimation*)

6.1 成本构成

成本估计是可行性研究中最易出错、也最困难的一部分。一般说来, 基于计算机的系统成本主要由四部分组成: [p.74]

- 购置并安装软件/硬件等有关设施的费用;
- 系统开发费用;
- 系统安装、运行和维护费用;
- 人员培训费用等。

6.2 GB/T36964-2018 软件开发成本度量规范

根据 GB/T36964-2018《软件工程——软件开发成本度量规范》, 软件开发成本包括软件开发过程中的所有人力成本和非人力成本之和。 [p.75]

直接人力成本 包括开发单位项目组成员的工资、奖金和福利等人力资源费用。非全职人员按其

在项目中投入的工作量占项目总工作量的比例折算。

间接人力成本 指开发单位服务于开发管理整体需求的非项目组人员的人力资源费用分摊, 例如

部门经理、项目管理办公室人员等的工资、奖金和福利。

直接非人力成本 包括办公费、差旅费等。

间接非人力成本 是指不为某个特定项目但服务于整体开发活动的非人力成本, 例如房租、水电、知识产权等费用。

6.3 成本估算方法分类

在可行性研究阶段, 估算项目成本及工作量通常可以采用以下几种方法: [p.76]

1. 基于已完成的类似项目进行估算——自顶向下 (类比法);
2. 使用简单的“分解技术”来进行成本及工作量的估算——由底向上;
3. 使用经验模型进行成本及工作量的估算。

6.4 自顶向下估算 (*Top-Down Estimation*)

依据先前已完成项目所耗费的总成本 (总工作量), 推算新开发软件的总成本 (总工作量), 然后在项目内部进行成本分配。 [p.78]

- **优点:** 工作量小, 速度快。
- **缺点:** 对开发中某些局部问题或特殊困难易低估, 甚至没有考虑。如果所开发的软件缺乏可借鉴的经验, 则估计偏差可能较大。

6.5 德尔菲法 (*Delphi Method*)

当参加估计的人数较多时, 可采用德尔菲 (Delphi) 法: [p.79]

1. 组织者将系统定义文件或规格说明发放给每位估计人员;
2. 估计人员通过研究系统定义文件规格说明, 依据自己以往的经验, 各自单独进行成本估算, 并填入有关表格, 说明有关理由;

3. 组织者综合估计人员的估计，并对个别与众不同的估计进行单独讨论，将估计结果分类摘要通知大家；
4. 重复上述估计过程，直到大家的意见接近为止（一般 3-5 轮）。

特点：匿名性、反馈性、统计性。

6.6 由底向上估算 (*Bottom-Up Estimation*)

在应用由底向上成本估计方法之前，估计者必须先了解待开发软件的范围： [p.82]

- 软件范围包括功能、性能、限制、接口和可靠性等；
- 在估算开始之前，应对软件范围进行适当的细化，以提供较详细的细节；
- 对于细化得到的任务单元可交给该任务的开发人员去估计，得到各任务单元的估计工作量和成本；
- 然后，将各任务单元的成本汇合成项目的总共工作量和成本。

注意：对涉及全局的花费（如质量管理）可能估计不足甚至完全忽视，使成本估计可能偏低。

6.6.1 由底向上估算示例

以一个计算机辅助设计（CAD）应用开发软件包为例： [p.83-84]

初步软件范围说明：

- 软件接收来自操作员的二维或三维几何数据；
- 操作员通过用户界面与系统进行交互并控制它们；
- 该用户界面具有良好的人机接口设计特征；
- 所有的几何数据和其他支持信息均保存在一个 CAD 数据库中；
- 开发一些设计分析模块以产生在各种图形设备上显示的输出；
- 软件在设计中要考虑与外设进行交互并控制它们，包括鼠标、数字化仪和激光打印机。

功能分解后识别的子系统：

1. 用户界面及控制（UICF）
2. 二维几何分析（2DGA）
3. 三维几何分析（3DGA）
4. 数据库管理（DBM）
5. 计算机图形显示（CGDF）
6. 外设控制（PC）
7. 设计分析模块（DAM）

对每一功能估算相应的代码行数，再根据历史数据得出每一功能的成本和相应的工作量，最后汇合成系统的总成本和总工作量。 [p.85]

6.7 经验模型估算 (*Empirical Model Estimation*)

由于经验数据是从一些有限的项目中得到的，而且软件类型和开发环境各不相同，因而模型中得到的结果必须慎重使用。 [p.87]

利用已完成项目的样本数据进行分析，从而建立有关经验公式来预测项目所需的成本、工作量等，具有比较客观、计算结果可重复（即无论何时使用模型，其结果相同）等优点。主要经验

模型如下：

- 静态单变量模型；
- 动态多变量模型；
- COCOMO 模型。

6.7.1 静态单变量模型 (*Static Single-Variable Model*)

利用被估软件单个特征的估计量。典型结构为： [p.88]

$$E = A + B \times (\text{估计变量})^C$$

其中 A 、 B 和 C 是由经验导出的常数； E 是以人月（PM）为单位的工作量；”估计变量”是被估软件特征的估计量，如代码行数等。

Walston 和 Felix 模型示例：

$$\begin{aligned} E &= 5.2 \times (\text{KLOC})^{0.91} \\ D &= 4.1 \times (\text{KLOC})^{0.36} = 13.47 \times E^{0.35} \\ \text{DOC} &= 49 \times (\text{KLOC})^{1.01} \\ S &= 0.54 \times E^{0.6} \end{aligned}$$

其中 KLOC 表示千行源代码行数，以一条机器指令为一行源代码。对于非机器指令编写的源程序，应转换成机器指令源代码行数来考虑。

软件单个特征的估算（如代码行数等）可以采用前述的德尔菲（Delphi）方法来估算。

6.7.2 动态多变量模型——Putnam 模型 (*Putnam Model*)

动态多变量模型把项目的资源需求看成是时间的函数。普特南（Putnam）依据大型软件工程项目（总工作量达到或超过 30 人年）的数据，提出 Putnam 模型： [p.89]

$$L = C_K \times K^{1/3} \times t_d^{4/3}$$

其中 L 表示源代码行数（以 LOC 计算）； K 表示软件全生存周期（含维护在内）所需工作量（以人年计）； t_d 表示项目开发持续时间（以年计）； C_K 表示技术状态常数。

由此可导出：

$$K = \frac{L^3}{C_K^3 \times t_d^4}, \quad t_d = \left[\frac{L^3}{C_K^3 \times K} \right]^{1/4}$$

这表明，如果缩短开发时间，将意味着增加项目的开发工作量。

规模在 100KLOC 以上的软件用这个模型是适当的。

6.8 COCOMO 模型 (*Constructive Cost Model*)

玻姆（Boehm）将软件成本估算分成 3 个由粗到细的层次：基本层、中间层和详细层。每个层次又按软件项目的应用领域和复杂程度分成 3 种类型：组织型、半独立型和嵌入型。 [p.90]

COCOMO 模型通用公式：

$$MM = a \times (KDSI)^b \times \prod_{i=1}^{15} f_i$$

$$TDEV = c \times (MM)^d$$

其中 MM 表示开发工作量（人月）；KDSI 表示源指令条数（千行）；TDEV 表示开发时间（月）； f_i ($i = 1 \sim 15$) 表示 15 项项目影响调节因子； a, c 表示模型系数； b, d 表示模型指数。

6.8.1 基本层 COCOMO 模型

不考虑成本影响调节因子，是对软件成本的一种宏观粗略估计，是一个静态单变量模型。
[p.91]

表 2: 基本层 COCOMO 模型参数

项目类型	工作量公式	开发时间公式
组织型 (<i>Organic</i>)	$MM = 2.4 \times (KDSI)^{1.05}$	$TDEV = 2.5 \times (MM)^{0.38}$
半独立型 (<i>Semi-Detached</i>)	$MM = 3.0 \times (KDSI)^{1.12}$	$TDEV = 2.5 \times (MM)^{0.35}$
嵌入型 (<i>Embedded</i>)	$MM = 3.6 \times (KDSI)^{1.20}$	$TDEV = 2.5 \times (MM)^{0.32}$

类型说明：

- **组织型**：较小、较简单的软件项目；
- **半独立型**：软件的需求介于”组织型”和”嵌入型”之间；
- **嵌入型**：必须在一组严格的硬件、软件及操作约束下开发的软件项目，对接口、数据结构、算法要求较高。

6.8.2 中间层 COCOMO 模型

主要考虑了从整个生存期来衡量成本影响调节因子，共 15 项，分成 4 类： [p.92]

- **产品属性**：软件可靠性、数据库规模、产品复杂度；
- **硬件属性**：执行时间约束、主存约束、虚拟机易变性、环境周转时间；
- **人员属性**：分析员能力、应用经验、程序员能力、虚拟机使用经验、编程语言使用经验；
- **项目属性**：现代编程实践、软件工具使用、开发进度约束。

示例：假设对前面所述的 CAD 应用开发软件包采用中间层 COCOMO 模型进行成本估算。由程序的规模（代码行数）和特征可看出该项目总体类型为组织型。如果分析员与程序员的工资平均每人每月按 15000 元计算，则该项目的开发人员的工资总额为： $120 \times 15000 = 180$ （万元）。
[p.94]

6.8.3 详细层 COCOMO 模型

需要考虑各调节因子对于不同开发阶段的影响。针对每一个影响因素，按模块级、子系统级和系统级，有三张工作量因素分级表，供不同级别的估算使用。 [p.95]

6.9 COCOMO II 模型

玻姆等于 2000 年提出 COCOMO II 模型。COCOMO II 模型可以处理较广泛的软件工程技术：面向对象、包含瀑布模型及其它生存期模型、复用、第四代语言等。 [p.96]

玻姆关于自己提出的 COCOMO 模型的评价意见是：

一个软件估算模型能够在成本估算上相差不到 20%，时间估算上相差不到 70%，就相当不错了。

当开发产品时，实际的开发工作量应与预测值进行比较。出现偏差可作为一个有问题的早期警告。管理者应该分析出现问题的原因，如开发小组不胜任、产品规模低估了等，并采取合适的行动使影响最小化。

6.10 软件开发成本度量规范——估算过程

根据国家标准 GB/T36964-2018，软件开发成本估算分为四个层层递进、逐步细化的过程： [p.99]

1. **规模估算**——确定软件的规模（功能点、代码行等）；
2. **工作量估算**——根据规模和基准数据计算工作量；
3. **成本估算**——将工作量转化为人力成本；
4. **总成本估算**——加上非人力成本和附加成本。

注意事项：需求模糊或不定时，采用类比或类推法估算工作量或成本；间接成本是否与工作量有关，取决于成本分摊计算方式。

6.11 软件规模估算 (*Software Size Estimation*)

软件规模估算是后续计算软件项目的工作量、成本和进度的主要输入。 [p.100]

在规模估算过程中需要注意：

- 在规模估算开始前，应根据可行性研究报告或类似文档明确项目需求及系统边界；
- 项目需求除包含最基本的业务需求外，还应进行初步的子系统/模块划分，并对每一子系统或模块的基本用户需求进行说明；
- 依据项目特点和需求详细程度不同，估算人员在选择估算方法时应采用纳入国际标准的功能点方法 (*Function Point Method*) 进行功能规模估算；
- 若当前的项目需求极其模糊或不明确，可不进行规模估算，而直接采用类比法或类推法估算工作量和成本。

6.12 功能点度量方法

五种功能点度量方法的国际/国内标准： [p.101]

- SJ/T11617—2016 软件工程 COSMIC-FFP：一种功能规模测量方法
- SJ/T11618—2016 软件工程 MK II 功能点分析计数实践指南
- SJ/T11619—2016 软件工程功能规模测量 NESMA 方法
- SJ/T11620—2016 信息技术软件和系统工程 FiSMA1.1 功能规模测量方法
- ISO/IEC 20926:2009 软件与系统工程软件测量 IFPUG 功能规模测量方法

IFPUG 标准和 NESMA 标准仍然是目前最主要的标准：

- 国际基准比对组织中超过 90% 的数据采用 IFPUG/NESMA 方法；
- 国内的行业数据则 100% 采用 IFPUG/NESMA 方法。

在适用 IFPUG 或 NESMA 方法时，可以根据需求的粒度和管理需要，选择预估功能点方法、估算功能点方法或者详细功能点方法。

6.13 工作量估算与成本计算

项目规模估算完成后即可获得总工时。可以根据软件行业基准数据来获取各类工作的工作量占比。 [p.102]

直接人力成本估算根据工作量与人月单价计算。间接人力成本宜按工作量比例进行分摊。 [p.103]

7 效益分析 (*Benefit Analysis*)

7.1 软件项目效益的两大组成部分

软件项目效益有两部分：经济效益和社会效益。

[p.109-110]

7.1.1 经济效益 (*Economic Benefits*)

经济效益是指使用新系统而增加的收入，包括使用新系统节省的运行费用，是一种**有形效益** (*Tangible Benefit*)。具体体现在：

- 提高生产效率
- 降低生产成本
- 提升产品质量
- 缩短生产周期
- 降低物资库存

7.1.2 社会效益 (*Social Benefits*)

社会效益是一种**无形效益** (*Intangible Benefit*)，主要从性质上、心理上进行衡量，很难直接量化，但在某些情况下，无形效益能转化成有形的效益。具体体现在：

- 推动技术创新
- 提升员工技能
- 改善工作环境
- 推动社会发展
- 加强单位管理

7.2 货币的时间价值 (*Time Value of Money*)

通常投资在前，取得效益在后，因此要考虑货币的时间价值。货币的时间价值常用利率的形式来表示。

[p.112]

设年利率为 i ，当前存入的货币数为 P (Present, 现值) 元，则 n 年后可得到的货币数为 F (Future, 终值) 元：

$$F = P \times (1 + i)^n$$

关键概念：

- P ：现值（或初始值）
- i ：计息期利率
- F ：终值（或本利和）
- n ：计息期数
- $(1 + i)^n$ ：复利终值系数，用符号 $(F/P, i, n)$ 表示，即 $F = P \times (F/P, i, n)$

贴现 (*Discounting*)：将未来某时间点资金的价值折算为现在时间点的价值。

$$P = \frac{F}{(1 + i)^n}$$

7.3 纯收入 (*Net Income*)

纯收入的值等于整个生存周期内系统的累积经济效益（折算成当前值）与投资之差。[p.113]

这相当于比较投资开发一个软件系统和把钱存在银行中（或贷给其他企业）这两种方案的优劣。从经济的观点来看，考虑到软件项目开发的风险，只有当纯收入大于零时才能考虑投资。

7.4 投资回收期 (*Payback Period*)

投资回收期也是衡量工程价值的一项经济指标，其值等于使累计的经济效益（折算成当前值）等于最初投资所需要的时间。[p.114]

投资回收期越短，就能越快获得经济效益，因而这项工程也就越值得投资。

7.5 投资回收期 (*Return on Investment, ROI*)

投资回收期可用来衡量投资效益的大小，并可以和银行年利率进行比较，在衡量工程的经济效益时是重要的参考数据。[p.115]

投资回收率的计算可以通过解下列方程得出：

$$P = \frac{F_1}{(1+j)} + \frac{F_2}{(1+j)^2} + \cdots + \frac{F_n}{(1+j)^n}$$

其中 P 表示当前的投资额， F_i 表示第 i 年年底的效益 ($i = 1, 2, \dots, n$)， n 表示系统的使用寿命， j 表示投资回收期。

7.6 经济效益分析综合示例

假定某项目开发的系统取代现有系统，可以每年节省 40 万元。项目开发投资 110 万。预计系统的寿命为 5 年，银行利率是 0.03。试对项目进行经济效益分析。[p.116]

表 3: 逐年效益折现计算

年份	计算过程	折现值 (万元)
第 1 年	40/1.03	38.8
第 2 年	40/(1.03 × 1.03)	37.7
第 3 年	40/(1.03 ³)	36.6
第 4 年	40/(1.03 ⁴)	35.5
第 5 年	40/(1.03 ⁵)	34.5

分析结果：

- 累计效益（折算成当前值）= 183.1 万元
- 纯收入 = 183.1 - 110 = 73.1 万元
- 投资回收期 = 2 + (110 - 38.8 - 37.7)/36.6 = 2.91 年
- 投资回收期：在 0.23 到 0.24 之间

7.7 效益分析总结

系统的效益分析随系统的特征而异： [p.117]

- 根据总投资的情况和年度效益分析的结果，可以进一步计算纯收入、投资回收期和投资回收率等；
- 管理者根据有关分析结果，结合其它潜在的对投资的使用，考虑有关社会效益，可以在经济上确定系统是否值得投资开发。

7.8 CIMS 项目效益分析实例

某大型企业 CIMS 项目的效益分析示例： [p.118]

(1) 直接经济效益（每年）总额约为 796 万元，其中包含：

- 实现以物资为主线的生产信息集成，合理制定物资储备定额，缩短物资库存周期，可降低库存 2%（平均库存周期从 30 天降到 24 天），按现有月末库存 1800 万元，年利率 6.75% 计算，每年可节约利息： $1800 \times 20\% \times 6.75\% = 24.3$ （万元）。

(2) 社会效益分析：

- 通过 CIMS 应用工程的实施，引入计算机集成制造系统，可加快公司员工更新观念的速度，加速公司人员素质的转变，为公司的人才培养战略提供良好的环境。
- CIMS 工程的实践将推动相关技术和管理创新。

8 本章要点总结

1. **可行性研究**是软件项目启动前的关键步骤，旨在以最小代价判定项目是否值得开发。研究内容包括技术可行性、经济可行性和社会可行性三大方面。
2. **系统分析**包括组织结构分析（组织结构图、业务联系图、业务功能树）、系统处理流程分析（系统流程图、泳道图）和系统数据流分析（DFD）。
3. **故障树分析（FTA）**是一种重要的系统可靠性分析方法，包括顶事件确定、边界条件设定、故障树构建与规范化、以及定性分析（最小割集）和定量分析（概率计算、重要度分析）。
4. **成本估算**方法包括自顶向下的类比法（德尔菲法）、由底向上的分解法以及经验模型法（CO-COMO 模型族、Putnam 模型、Walston-Felix 模型等）。
5. **效益分析**综合考虑经济效益（有形）和社会效益（无形），运用货币时间价值理念计算纯收入、投资回收期和投资回收率等关键指标。